

Ứng dụng của cây đoạn

Lin Tao

Nguồn gốc: Applications of segment trees

IOI China candidate team papers / 2004 / Lin Tao.pdf

Tóm tắt

Trong giải bài thi lập trình, ta thường gặp các thao tác liên quan đến đoạn hoặc khoảng, chẳng hạn tính diện tích hợp của nhiều hình chữ nhật, ghi lại giá trị lớn nhất, nhỏ nhất hoặc tổng lượng trên một đoạn, rồi duy trì các giá trị này khi chèn, xóa và sửa đoạn.

Cây đoạn có cấu trúc nhị phân dạng cây rất tốt, nên có thể hoàn thành các thao tác ấy một cách hiệu quả. Bài viết này giới thiệu nhiều thao tác trên cây đoạn và một số mở rộng của nó.

Thông qua ba ví dụ: *Snake*, *Cuboid* và *Battlefield Statistics System*, bài viết trình bày các thao tác chèn, xóa, tìm kiếm cơ bản trên cây đoạn, các thao tác sửa và xóa không đều, cũng như cách mở rộng sang hai chiều.

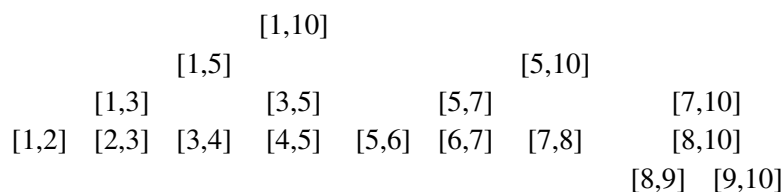
Từ khóa. Cây đoạn; chia đôi; co cây con; bung lá; cây diện tích.

1 Định nghĩa và đặc trưng của cây đoạn

Định nghĩa 1: cây đoạn. Cây đoạn là một cây nhị phân, ký hiệu là $T(a, b)$, trong đó hai tham số a, b cho biết nút đang biểu diễn đoạn $[a, b]$. Gọi độ dài của đoạn là $L = b - a$. Ta định nghĩa đệ quy $T[a, b]$ như sau:

- Nếu $L > 1$, đoạn $[a, \lfloor (a + b)/2 \rfloor]$ là con trái của T , còn đoạn $[\lfloor (a + b)/2 \rfloor, b]$ là con phải của T .
- Nếu $L = 1$, T là một nút lá.

Ví dụ, cây đoạn biểu diễn đoạn $[1, 10]$ có thể hình dung như sau:



Trong phần dưới đây, mọi giá trị logarit đều được làm tròn lên.

Định lý 1. Cây đoạn chia một đoạn bất kỳ nằm trên đoạn ban đầu thành không quá $2 \log L$ đoạn con.

Chứng minh. Trước hết xét đoạn (a, b) và một đoạn con (c, d) . Nếu $c \leq a$ hoặc $d \geq b$, thì trong (a, b) , đoạn này bị chia thành không quá $\log(b - a)$ đoạn. Ta chứng minh bằng quy nạp. Với đoạn đơn vị, nhiều nhất chỉ có một đoạn, nên mệnh đề đúng. Giả sử mệnh đề đúng với con trái và con phải của (a, b) . Khi $c \leq a$:

1. Nếu $d \leq (a + b) \div 2$, việc chia đoạn tương đương với chia trong con trái. Số đoạn không vượt quá $\log((a + b) \div 2 - a)$, do đó không vượt quá $\log(b - a)$.
2. Nếu $d > (a + b) \div 2$, đoạn này được chia thành đoạn do con trái biểu diễn cộng với phần được chia trong con phải. Số đoạn không vượt quá $1 + \log(b - (a + b) \div 2)$, cũng không vượt quá $\log(b - a)$.

Trường hợp $d \geq b$ được chứng minh tương tự.

Bây giờ xét một đoạn bất kỳ trong (a, b) và vẫn dùng quy nạp. Với đoạn đơn vị, nhiều nhất có một đoạn. Giả sử mệnh đề đúng với cả con trái và con phải của (a, b) . Với đoạn (c, d) :

1. Nếu $d \leq (a + b) \div 2$, số đoạn bằng số đoạn sinh ra trong con trái, nhỏ hơn $\log((a + b) \div 2 - a) < 2 \log(b - a)$.
2. Nếu $c > (a + b) \div 2$, số đoạn bằng số đoạn sinh ra trong con phải, nhỏ hơn $\log(b - (a + b) \div 2) < 2 \log(b - a)$.
3. Nếu cả hai trường hợp trên đều không xảy ra, đoạn này cắt qua biên giữa hai con. Ở con trái, nó thuộc trường hợp $d > V.Lson.b$ nên số đoạn không vượt quá $\log(b - (a + b) \div 2)$. Ở con phải, nó thuộc trường hợp $c \leq V.Rson.a$ nên số đoạn không vượt quá $\log((a + b) \div 2 - 1)$. Vì vậy tổng số đoạn trong (a, b) không vượt quá $2 \log(b - a)$.

Kết luận này là cơ sở lý thuyết để cây đoạn thực hiện chèn, xóa, tìm kiếm một đoạn trong thời gian $O(\log L)$.

2 Ví dụ 1: Snake

*Snake*¹ cho N điểm trên mặt phẳng. Cần tìm một số đoạn thẳng thỏa các yêu cầu:

1. các đoạn thẳng phải tạo thành một đường khép kín;
2. đầu mút của các đoạn chỉ được là N điểm đã cho;
3. hai đoạn thẳng gặp nhau tại một điểm phải vuông góc với nhau;
4. mọi đoạn thẳng đều song song với một trục tọa độ;
5. ngoài N điểm đã cho, các đoạn thẳng không được tự cắt;
6. tổng độ dài của tất cả đoạn thẳng phải nhỏ nhất.

Nếu tồn tại các đoạn như vậy thì in ra độ dài nhỏ nhất, ngược lại in ra 0.

Phân tích

Ta bắt đầu từ yêu cầu của đề: trước hết dựng hình thỏa điều kiện, sau đó giải quyết bài toán tổng độ dài nhỏ nhất.

Đề bài rõ ràng yêu cầu một đa giác có N đỉnh là N điểm đã cho. Mọi đoạn đều song song với trục tọa độ, nên mỗi điểm chỉ có thể nối với các điểm ở bốn hướng trên, dưới, trái, phải. Vì hai đoạn kề tại một

¹Saratov State University Problem Archive, 1028, *Snake*. Bài này kiểm tra các thao tác chèn, xóa và tìm kiếm cơ bản.

điểm phải vuông góc, mỗi đỉnh phải nối với đúng một đoạn song song trục X và đúng một đoạn song song trục Y .

Sau khi sắp xếp các điểm, xét những điểm cùng nằm trên một đường ngang, giả sử chúng là $P_1, P_2, P_3, P_4, \dots, P_n$. Nếu P_1 cần một đoạn song song trục X , nó bắt buộc phải nối với điểm bên phải gần nhất là P_2 . Khi đó P_3 không thể nối tiếp với P_2 , vì P_2 sẽ có hai đoạn song song trục X kề với nó; vì vậy P_3 chỉ có thể nối với P_4 , P_5 nối với P_6 , v.v. Lập luận trên đường thẳng đứng cũng tương tự. Do đó cách dựng đoạn là duy nhất, và bài toán độ dài nhỏ nhất cũng được quyết định theo cách dựng này.

Vì nghiệm là duy nhất, còn việc kiểm tra các đoạn có liên thông hay không có thể làm bằng tìm kiếm theo chiều rộng, điểm mấu chốt còn lại là kiểm tra hình thu được có hợp lệ hay không, tức là các đoạn không tự cắt ngoài N điểm đã cho. Có ba khả năng trực quan: không liên thông là không hợp lệ; liên thông và không tự cắt là hợp lệ; có tự cắt là không hợp lệ.

Mọi đoạn đều song song với một trong hai trục, nên bài toán có tính chất đoạn rất rõ. Ta có thể dùng cây đoạn để kiểm tra tự cắt.

Chỉ đoạn song song trục X mới có thể cắt đoạn song song trục Y , vì vậy chỉ cần xét với mỗi đoạn thẳng đứng xem có đoạn ngang nào cắt nó hay không. Nếu đoạn $(x, y_1) - (x, y_2)$ cắt đoạn $(x_1, y) - (x_2, y)$, ta phải có

$$x_1 < x < x_2, \quad y_1 < y < y_2.$$

Từ điều kiện $x_1 < x < x_2$, ta sắp xếp mọi sự kiện theo tọa độ X . Cần chú ý rằng hai đoạn gặp nhau tại đầu mút không được xem là tự cắt, nên khi nhiều sự kiện có cùng tọa độ X , phải chọn thứ tự đúng: đầu phải của đoạn ngang trước, sau đó đến đoạn thẳng đứng, cuối cùng là đầu trái của đoạn ngang.

Ta xây cây đoạn trên miền tọa độ Y . Sau khi sắp xếp, mỗi đoạn hoặc mỗi đầu mút của đoạn được xem là một sự kiện. Quét các sự kiện theo X tăng dần. Khi gặp đầu trái của một đoạn ngang, xem tọa độ Y của nó là một điểm và chèn điểm này vào cây đoạn trên trục Y . Khi gặp đầu phải của một đoạn ngang, xóa điểm tương ứng khỏi cây đoạn. Khi gặp đoạn thẳng đứng $L_1 = (x, y_1) - (x, y_2)$, giả sử có đoạn ngang $L_2 = (x_1, y) - (x_2, y)$ cắt nó. Do $x_1 < x < x_2$, đầu trái của L_2 đã được quét qua, nên điểm đại diện của L_2 đã được chèn vào cây; đầu phải của L_2 chưa được quét qua, nên điểm ấy vẫn còn trong cây. Nói cách khác, mọi điểm còn trong cây đều thỏa $x_1 < x < x_2$. Để kiểm tra điều kiện $y_1 < y < y_2$, ta chỉ cần tìm trong cây xem đoạn $[y_1 + 1, y_2 - 1]$ có điểm nào hay không, vì giao tại đầu mút được phép. Nếu có, hình đã tự cắt và không hợp lệ.

Khi cài đặt, mỗi nút cây đoạn cần thêm một biến ghi số điểm trong đoạn mà nút đó biểu diễn. Đơn vị của cây đoạn, tức nút lá, được đổi thành một điểm chứ không phải một đoạn đơn vị. Sau mỗi lần chèn hoặc xóa, chỉ cần cập nhật biến trên các nút liên quan. Mã giả cho chèn, xóa và tìm kiếm nằm trong Phụ lục 1.

Sau khi rời rạc hóa tọa độ Y , mỗi thao tác trên có độ phức tạp $O(\log n)$. Vì số đoạn là $O(n)$, toàn bộ bài toán có độ phức tạp $O(n \log n)$. Nếu mở rộng bài toán thành đếm mọi giao điểm của các đoạn, ta chỉ cần trong bước tìm kiếm thống kê tất cả điểm nằm trong đoạn cần hỏi.

3 Ví dụ 2: Cuboid

*Cuboid*² xét một hệ tọa độ dương ba chiều có $N \leq 5000$ điểm. Cần tìm một điểm $P(x, y, z)$ sao cho hình hộp chữ nhật do hai đỉnh đối diện $O(0, 0, 0)$ và $P(x, y, z)$ xác định không chứa bất kỳ điểm nào trong N điểm đã cho; điểm nằm trên biên hình hộp không được xem là bị chứa. Thể tích hình hộp phải lớn nhất, và mọi tọa độ x, y, z đều không vượt quá 1000000.

²Polish Olympiad in Informatics 1999, *Cuboid*. Bài này kiểm tra thao tác sửa dữ liệu theo khối lớn.

Phân tích

Hình hộp do $P(x, y, z)$ biểu diễn chứa một điểm Q khi mọi tọa độ của P đều lớn hơn tọa độ tương ứng của Q , tức là

$$P_x > Q_x, \quad P_y > Q_y, \quad P_z > Q_z.$$

Với hình hộp có thể tích lớn nhất, tọa độ của P trên bất kỳ trục nào cũng phải bằng tọa độ tương ứng của một trong N điểm, hoặc bằng biên. Nếu không, ta có thể tăng tọa độ đó đến giá trị nhỏ nhất lớn hơn nó; thể tích tăng mà vẫn không chứa thêm điểm nào.

Giả sử tọa độ X của P đã được cố định. Sắp xếp mọi tọa độ Y của các điểm, thu được dãy $y[1], y[2], y[3], \dots$. Đặt mảng max , trong đó $max[i]$ là giá trị lớn nhất có thể của tọa độ Z khi tọa độ Y của P bằng $y[i]$. Khi tọa độ Y tăng lên, ràng buộc lên tọa độ Z nhiều hơn, nên mảng max là đơn điệu không tăng.

Sắp xếp mọi điểm theo tọa độ X . Khi tọa độ X của P bằng tọa độ của điểm thứ i , các điểm từ i trở đi không thể bị P chứa. Ta liệt kê tọa độ X của P từ nhỏ đến lớn, đồng thời liên tục duy trì mảng max . Khi tọa độ X của P chuyển từ điểm thứ i sang điểm thứ $i + 1$, tọa độ của điểm thứ i sẽ tạo thêm một ràng buộc lên mảng max . Xét ràng buộc do điểm thứ i , ký hiệu $Q(x, y, z)$, tạo ra: những vị trí có tọa độ Y nhỏ hơn Q_y hoặc có giới hạn Z đã không vượt quá Q_z thì không cần sửa. Thực chất ta chỉ cần đổi giá trị max trên một đoạn $[i, j]$ thành Q_z . Để thao tác đoạn này hiệu quả, ta dùng cây đoạn.

Ta xây cây đoạn trên mảng max . Trong quá trình liệt kê tọa độ X của P , mỗi ràng buộc mới của một điểm tương đương với việc sửa các giá trị trong một đoạn của cây. Nếu đoạn cần sửa chứa toàn bộ đoạn do nút V biểu diễn, lẽ ra V và mọi con của V đều phải được sửa. Để tránh các thao tác lặp vô ích, ta chỉ sửa V : vì mọi giá trị trong đoạn của V đã giống nhau, ta co cả cây con V thành một nút lá. Ngược lại, nếu đoạn cần sửa không chứa toàn bộ đoạn của V trong khi V đã bị co thành một lá, ta bung lá này ra thành hai con để tiếp tục sửa cục bộ.

Thể tích cần tìm có dạng

$$P_x \cdot max[i] \cdot y[i].$$

Với P_x hiện tại đã biết, điểm mấu chốt là giá trị lớn nhất của $max[i] \cdot y[i]$. Vì vậy trong cây đoạn, mỗi nút cần ghi giá trị lớn nhất của $max[i] \cdot y[i]$ trên đoạn nó biểu diễn, và duy trì giá trị ấy khi mảng bị sửa. Giá trị lớn nhất ở gốc nhân với tọa độ X hiện tại của P chính là thể tích lớn nhất ở bước đó. Ngoài ra, vì $P_x, P_y, P_z \leq 1000000$, cần thêm các điểm biên $(0, 0, 1000000)$, $(0, 1000000, 0)$ và $(1000000, 0, 0)$. Mã giả cho thao tác sửa nằm trong Phụ lục 2.

Tổng kết thuật toán

Nhờ dùng kỹ thuật co cây con, ta tránh được thao tác lặp. Sau khi rời rạc hóa, số nút bị sửa trong mỗi lần là $O(\log n)$, và việc duy trì các giá trị liên quan cũng ở mức này. Do đó độ phức tạp của bài toán là $O(n \log n)$.

Điểm thành công của bài này là không xem cây đoạn một cách cứng nhắc như một cấu trúc cố định. Ta nắm bản chất của nút lá trong cây đoạn: dữ liệu trong đoạn là đồng nhất. Tùy tình huống, ta co cây con thành lá hoặc bung lá ra thành các con, nhờ đó tránh được các thao tác trùng lặp.

4 Ví dụ 3: Battlefield Statistics System

Năm 2050, cuộc chiến giữa loài người và người ngoài hành tinh đã trở nên vô cùng ác liệt. Đúng lúc đó, loài người phát minh ra một loại siêu vũ khí có thể tấn công đồng thời nhiều mục tiêu kẻ nhau. Mọi đơn vị ngoài hành tinh có phòng thủ nhỏ hơn hoặc bằng sức công phá của vũ khí này sẽ bị tiêu diệt. Tuy

nhiên, chỉ có siêu vũ khí là chưa đủ; loài người còn cần một hệ thống thông kê chiến trường để liên tục phản hồi thông tin về quân đội ngoài hành tinh. Nhiệm vụ khó khăn này được giao cho bạn. Hãy nhanh chóng thiết kế một hệ thống như vậy.

Hệ thống cần xử lý hai loại thông tin:

1. Người ngoài hành tinh tăng viện một đơn vị có phòng thủ v tại mỗi vị trí trong đoạn $[x_1, x_2]$.
2. Loài người dùng siêu vũ khí tấn công mọi vị trí trong đoạn $[x_1, x_2]$ với sức công phá v ; mọi đơn vị có phòng thủ không vượt quá v bị tiêu diệt. Hệ thống phải trả về số người ngoài hành tinh bị tiêu diệt trong lần tấn công này.

Ghi chú: một đơn vị có phòng thủ i gồm i người ngoài hành tinh, trong đó người thứ j có phòng thủ j . Giới hạn dữ liệu là $1 \leq x_1 \leq x_2 \leq 1000$, $v \leq 1000$, và tổng số thông tin $m \leq 2000$.

Phân tích

Trước hết trừu tượng hóa hai loại thông tin. Dùng mảng hai chiều $T[x, y]$ để biểu diễn số đơn vị có phòng thủ y trên đoạn đơn vị $[x, x]$. Nếu tăng thêm đơn vị có phòng thủ v trên đoạn $[x_1, x_2]$, ta thêm vào mảng hai chiều một hình chữ nhật $[x_1, x_2] \times [1, v]$. Một lần dùng vũ khí tương đương với việc tìm số đơn vị trong hình chữ nhật $[x_1, x_2] \times [1, v]$, rồi xóa tất cả đơn vị trong hình chữ nhật đó. Vì vậy đây là một bài xử lý dữ liệu, cần hoàn thành hiệu quả hai thao tác:

1. thêm một hình chữ nhật vào mảng hai chiều;
2. tìm số đơn vị trong một hình chữ nhật và xóa chúng.

Nếu dùng cách đơn giản nhất, tức thao tác trên từng điểm của hình chữ nhật, độ phức tạp lên tới $O(mnv)$.

Trước hết có thể tối ưu theo một chiều: xử lý độc lập từng đoạn đơn vị $[x, x]$, và trên mỗi đoạn đơn vị dùng một cây đoạn để ghi số đơn vị của từng mức phòng thủ.

Khi chèn một hình chữ nhật $[x_1, x_2] \times [1, v]$, thao tác này trở thành $x_2 - x_1 + 1$ lần chèn đoạn $[1, v]$ trên cây đoạn, giống thao tác chèn thông thường.

Khi thống kê số đơn vị trong hình chữ nhật $[x_1, x_2] \times [1, v]$ và xóa chúng, ta cũng thực hiện $x_2 - x_1 + 1$ lần thống kê rồi xóa đoạn $[1, v]$ trên cây đoạn. Phần tìm kiếm khá đơn giản: mỗi nút ghi số lần đoạn của nó được phủ t và tổng số đơn vị *total* trong đoạn đó:

$$V.total = V.t \cdot (V.b - V.a + 1) + V.Lson.total + V.Rson.total.$$

Khó hơn là xóa các đơn vị trong đoạn, vì đoạn bị xóa không nhất thiết trùng với các đoạn từng được chèn. Giả sử cần xóa nút V và các con của nó; thao tác này tương đương với đặt mọi bản ghi trong V và các con về 0. Ta lại dùng kỹ thuật sửa dữ liệu khối lớn đã nhắc tới ở trên: co cây con. Trong nút V , thêm một biến Boolean ghi V có phải là lá hay không. Muốn xóa V thì đặt biến lá là đúng. Mặt khác, nếu cần xóa đoạn $[1, v]$ bên trong đoạn của V nhưng $[1, v]$ không phủ toàn bộ V , các đoạn đã chèn trước đây vốn phủ V phải được tách ra; ta phân phối phần phủ của V xuống hai con. Mã giả cho thao tác xóa nằm trong Phụ lục 3. Ngược lại, khi chèn $[1, v]$ vào trong đoạn của V , nếu đoạn V không bị phủ hoàn toàn và V hiện là một lá, ta bung V thành hai con rồi chèn $[1, v]$ vào hai con đó.

Sau tối ưu này, mỗi lần xóa, thống kê hoặc chèn có độ phức tạp $O(n \log v)$, nên tổng độ phức tạp giảm xuống $O(mn \log v)$.

Cải tiến thuật toán

Vì đây là một đoạn hai chiều, ta có thể xây dựng trên toàn bộ miền hai chiều một cấu trúc giống cây đoạn, gọi là **cây diện tích**. Giả sử miền hai chiều là $[1, 2^k] \times [1, 2^k]$. Ký hiệu $T(x_1, x_2, y_1, y_2)$ là cây diện tích biểu diễn hình chữ nhật $[x_1, x_2] \times [y_1, y_2]$. Ta định nghĩa đệ quy cây T như sau:

- Nếu $x_1 = x_2$ và $y_1 = y_2$, T là một nút lá.
- Nếu $x_2 - x_1 \leq y_2 - y_1$, hai con của T lần lượt là

$$T_1(x_1, \lfloor (x_1 + x_2)/2 \rfloor, y_1, y_2), \quad T_2(\lfloor (x_1 + x_2)/2 \rfloor + 1, x_2, y_1, y_2).$$

- Nếu $x_2 - x_1 > y_2 - y_1$, hai con của T lần lượt là

$$T_1(x_1, x_2, y_1, \lfloor (y_1 + y_2)/2 \rfloor), \quad T_2(x_1, x_2, \lfloor (y_1 + y_2)/2 \rfloor + 1, y_2).$$

Cách xây cây này thực chất tương đương với cây tứ phân quen thuộc, nhưng giảm số con để thao tác thuận tiện hơn. Vì một hình chữ nhật bất kỳ được cây tứ phân chia thành không quá $2k$ khối, nên với cách xây trên, một hình chữ nhật bất kỳ cũng được cây diện tích chia thành không quá $2k$ khối. Do đó trong cây này, một thao tác chèn hoặc xóa có độ phức tạp không vượt quá $O(2k)$. Cách thao tác cụ thể tương tự cây đoạn một chiều nên không nhắc lại. Tổng độ phức tạp giảm xuống $O(m(n + v))$.

5 Tổng kết

Từ ba bài trên, có thể rút ra phương pháp chung khi dùng cây đoạn để giải bài:

1. Phân tích đề để nhận ra bài toán có tính chất đoạn rõ rệt.
2. Theo yêu cầu bài toán, xây cây đoạn trên một miền thích hợp. Những bài thông thường thường cần rời rạc hóa tọa độ. Khi xây cây, đừng bị cái tên “cây đoạn” bó buộc đến mức chỉ xây trên đoạn thẳng; chỉ cần đối tượng biểu diễn được một đoạn và đoạn đó được tạo từ các đơn vị cơ bản, như một điểm, một đoạn, hoặc một giá trị trong mảng, thì đều có thể xây cây đoạn. Cũng đừng bị giới hạn ở một chiều; tùy đề bài có thể xây cây diện tích, cây thể tích, v.v.
3. Với mỗi nút của cây, đặt các biến cần thiết để ghi giá trị mà bài toán yêu cầu.
4. Dùng cấu trúc cây để duy trì các biến ấy. Nếu cần tổng, giá trị của nút là tổng của hai con cộng với phần của chính nút; nếu cần cực trị, giá trị của nút được suy ra từ cực trị của hai con kết hợp với thông tin trên đoạn hiện tại. Lợi dụng đặc điểm mỗi lần chèn hoặc xóa chỉ sửa $O(\log L)$ nút, ta duy trì các biến liên quan sau sửa đổi trong thời gian $O(\log L)$.
5. Với các thao tác xóa không đều và sửa dữ liệu khối lớn, cần linh hoạt dùng cơ cây con và bung nút lá để tránh thao tác lặp.

Phụ lục

1. Chèn, xóa và tìm điểm trên cây đoạn

```
Procedure Insert(y, V) {chen diem y vao doan V}
Begin
  If V.a = V.b then Inc(V.t)
  Else Begin
```

```

    If y <= (V.a + V.b) div 2 then Insert(y, V.Lson);
    If y > (V.a + V.b) div 2 then Insert(y, V.Rson);
    V.t := V.Lson.t + V.Rson.t;
End;
End;

Procedure Delete(y, V) {xoa diem y khoi doan V}
Begin
    If V.a = V.b then Dec(V.t)
    Else Begin
        If y <= (V.a + V.b) div 2 then Delete(y, V.Lson);
        If y > (V.a + V.b) div 2 then Delete(y, V.Rson);
        V.t := V.Lson.t + V.Rson.t;
    End;
End;

Function Find(y1, y2, V): boolean
    {kiem tra trong doan V co diem nao thuoc [y1,y2] hay khong}
Begin
    If y1 <= V.a <= V.b <= y2 then
        Find := (V.t <> 0)
    Else Begin
        Find := false;
        If y1 <= (V.a + V.b) div 2 then
            If Find(y1, y2, V.Lson) then Find := true;
        If y2 > (V.a + V.b) div 2 then
            If Find(y1, y2, V.Rson) then Find := true;
    End;
End;

```

2. Sửa đoạn trong bài Cuboid

```

Procedure Modify(qx, qy, qz, V, a, b)
    {them rang buoc do diem (qx,qy,qz) len doan V[a,b]}
Begin
    {V.l = max[V.a], V.r = max[V.b]}
    If (y[a] > qy) and (V.r > qz) then Begin
        {co cay con V}
        V.leaf := true;
        V.l := qz; V.r := qz;
        V.maxS := qz * y[b];
        Exit;
    End;

    If (y[b] <= qy) or (V.l <= qz) then Exit;

    If V.leaf then Begin
        {bung la V thanh hai con, cac thuoc tinh ban dau giong V}
    End;
End;

```

```

V.Lson := V;
V.Lson.maxS := y[(a+b) div 2] * V.Lson.l;
V.Rson := V;
V.Rson.maxS := y[b] * V.Rson.l;
V.leaf := false;
End;

Modify(qx, qy, qz, V.Lson, a, (a+b) div 2);
Modify(qx, qy, qz, V.Rson, (a+b) div 2 + 1, b);
V.maxS := maximum(V.Lson.maxS, V.Rson.maxS);
End;

```

3. Xóa không đều trong bài thống kê chiến trường

```

Procedure Delete(y1, y2, V)
Begin
  If y1 <= V.a <= V.b <= y2 then Begin
    V.leaf := true; {bien V thanh nut la}
    V.t := 0;
    V.total := 0;
  End
  Else if not V.leaf then Begin
    {neu V da la la thi doan nay rong, khong can thao tac}
    Inc(V.Lson.t, V.t); {phan phoi phan phu V cho hai con}
    Inc(V.Rson.t, V.t);
    V.t := 0;
    If y1 <= (V.a + V.b) div 2 then Delete(y1, y2, V.Lson);
    If y2 > (V.a + V.b) div 2 then Delete(y1, y2, V.Rson);
    V.total := V.Lson.total + V.Rson.total;
  End;
End;

```